Guía de arranque y pruebas

Prueba técnica ITX · Back-end (Java / Spring Boot) y Front-end (React)

Esta guía explica, paso a paso y en Windows (PowerShell), cómo levantar la infraestructura, arrancar las dos aplicaciones, probarlas a mano y ejecutar todos los tests automáticos. Es el mismo documento en los dos repositorios.

Repositorio	Contenido	Puerto
prueba-itx-back	API REST GET `/product/{productId}/similar` con Spring Boot WebFlux, protegida con JWT de Keycloak. Incluye mocks, Keycloak, test de rendimiento k6 y tests e2e con Karate.	5000
prueba-itx-front	SPA en React + Vite para comprar móviles (listado y detalle), con login OIDC contra Keycloak. Consume el API pública de la prueba (itx-frontend-test.onrender.com).	5173

Contenido

1. Requisitos
2. Obtener el código
3. Paso 1 · Infraestructura con Docker
4. Paso 2 · Arrancar el back
5. Paso 3 · Probar el back a mano
6. Paso 4 · Arrancar y probar el front
7. Tests del back
8. Tests del front
9. Resumen de comandos
10. Problemas frecuentes
11. Parar todo

1. Requisitos

Herramienta	Versión	Comprobar con
Git	cualquiera reciente	`git --version`
JDK	**21** (o 17). Karate no funciona con Java 22 o superior	`java -version`
Maven	3.8 o superior	`mvn -v`
Node.js / npm	Node 18+ / npm 9+	`node -v` · `npm -v`
Docker Desktop	abierto y con el motor arrancado (Engine running)	`docker info`

Si tienes varios JDK instalados, comprueba en `mvn -v` la línea *Java version*: es el JDK que usa Maven de verdad. Para usar el 21 en la ventana de PowerShell actual:

```
$env:JAVA_HOME = "C:\Program Files\Java\jdk-21"
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
mvn -v      # debe mostrar: Java version: 21...
```

> **Nota.** Este cambio sólo afecta a esa ventana de PowerShell. Para hacerlo permanente, cambia JAVA_HOME en las variables de entorno de Windows.

2. Obtener el código

Clona los dos repositorios en la misma carpeta:

```
cd C:\Users\<usuario>\Documents
git clone <url-repositorio-back>  prueba-itx-back
git clone <url-repositorio-front> prueba-itx-front
```

3. Paso 1 · Infraestructura con Docker

Desde el repositorio de back se levantan los mocks de las APIs existentes, Keycloak y las herramientas del test de rendimiento:

```
cd prueba-itx-back
docker compose up -d simulado influxdb grafana keycloak
```

Servicio	URL	Para qué
simulado	http://localhost:3001	Mocks de las APIs existentes (ids similares y detalle de producto)
keycloak	http://localhost:8080	Servidor de identidad: emite los JWT del back y gestiona el login del front
influxdb	http://localhost:8086	Almacena las métricas del test de rendimiento
grafana	http://localhost:3000	Panel con los resultados del test de rendimiento

Comprueba que todo responde:

1. Mocks: abre `http://localhost:3001/product/1/similarids` → debe devolver `[2,3,4]`.

2. Keycloak (tarda 20-30 s la primera vez): abre `http://localhost:8080/realms/itx` → debe devolver un JSON con `"realm": "itx"`.

Credenciales de desarrollo (importadas desde `keycloak/itx-realm.json`):

Uso	Usuario / cliente	Contraseña / secreto
Login en el front	`demo`	`demo`
Token para el back (client credentials)	`similar-products-client`	`similar-products-secret`
Consola de administración (http://localhost:8080/admin)	`admin`	`admin`

> **Nota.** El aviso «the attribute version is obsolete» al ejecutar docker compose es inofensivo: viene del docker-compose.yaml original de la prueba.

4. Paso 2 · Arrancar el back

Opción A · Terminal

```
cd prueba-itx-back
mvn spring-boot:run
```

Opción B · IntelliJ IDEA: crea una configuración *Application* con la clase principal `com.itx.similarproducts.SimilarProductsApplication` y JDK 21, y pulsa Run.

La aplicación está lista cuando en la consola aparece `Netty started on port 5000`.

Opción C · Docker (no necesita Java ni Maven en local):

```
docker compose up -d --build simulado keycloak similar-products
```

5. Paso 3 · Probar el back a mano

El endpoint exige un token de Keycloak. En PowerShell:

```
$token = (Invoke-RestMethod -Method Post `
  -Uri http://localhost:8080/realms/itx/protocol/openid-connect/token `
  -Body @{ grant_type    = 'client_credentials'
           client_id     = 'similar-products-client'
           client_secret = 'similar-products-secret' }).access_token
$h = @{ Authorization = "Bearer $token" }

Invoke-RestMethod http://localhost:5000/product/1/similar -Headers $h
```

Petición	Resultado esperado	Por qué
`/product/1/similar`	Dress, Blazer, Boots	Caso normal, en orden de similitud
`/product/4/similar`	Shirt, Dress	El similar 5 no existe (404): se omite
`/product/5/similar`	Shirt, Dress	El similar 6 falla (500): se omite
`/product/2/similar`	1ª vez: Blazer, Trousers · 2ª vez (a los pocos segundos): + Coat	Coat tarda 5 s: se omite y se cachea en segundo plano
`/product/3/similar`	Trousers, y Coat si ya está en caché (responde en menos de 4 s)	Similares muy lentos (5 s y 50 s)
`/product/999/similar`	404	El producto no existe
Sin cabecera Authorization	401	Falta el token
`Authorization: Bearer token-falso`	401	Token inválido

Ejemplos de llamadas sin token o con token falso:

```
Invoke-RestMethod http://localhost:5000/product/1/similar
Invoke-RestMethod http://localhost:5000/product/1/similar `
  -Headers @{ Authorization = "Bearer token-falso" }
```

> **Nota.** El token caduca a los 5 minutos. Si empiezas a recibir 401 con el token bueno, vuelve a ejecutar la primera instrucción para pedir uno nuevo.

Con bash / curl:

```
TOKEN=$(curl -s -X POST http://localhost:8080/realms/itx/protocol/openid-connect/token \
  -d grant_type=client_credentials -d client_id=similar-products-client \
  -d client_secret=similar-products-secret | jq -r .access_token)
curl -H "Authorization: Bearer $TOKEN" http://localhost:5000/product/1/similar
```

6. Paso 4 · Arrancar y probar el front

Necesita Keycloak arrancado (paso 1). No hace falta el back: el front usa el API pública de la prueba.

```
cd prueba-itx-front
npm install
npm start
```

1. Abre `http://localhost:5173`: te redirige al login de Keycloak.

2. Entra con usuario `demo` y contraseña `demo`.

3. La primera carga del listado puede tardar hasta un minuto: el API está en Render (plan gratuito) y se «despierta» con la primera petición.

> **Nota.** Si ya tienes Keycloak levantado desde el back, no ejecutes el docker-compose.yml del front: ambos usan el puerto 8080. El del front sirve para usarlo sin el repositorio de back (`docker compose up -d keycloak`).

Qué comprobar

Vista	Comprobación
Login	Sin sesión redirige a Keycloak; tras entrar aparece «Hola, Demo» y el botón «Cerrar sesión», que vuelve al login.
Listado	Imagen, marca, modelo y precio; máximo 4 por fila y adaptable al ancho; el buscador filtra por marca y modelo mientras se escribe.
Detalle	Dos columnas (imagen | descripción y acciones); breadcrumb «Inicio / Marca Modelo»; enlace «Volver al listado»; atributos CPU, RAM, sistema operativo, pantalla, batería, cámaras, dimensiones y peso.
Acciones	Almacenamiento y color seleccionados por defecto si sólo hay una opción; «Añadir a la cesta» actualiza el contador de la cabecera.
Persistencia	Al recargar (F5) se mantiene el contador de la cesta y, en F12 → Network, no se repiten las peticiones al API durante 1 hora (F12 → Application → Local Storage, claves `itx-cache:`).

7. Tests del back

Nivel	Comando	Necesita
Unitarios (*Test)	`mvn test`	Nada
Integración (*IT)	`mvn verify`	Docker para KeycloakSecurityIT (sin Docker se omite)
End-to-end (Karate)	`mvn -f e2e/karate/pom.xml test`	Infraestructura + back arrancados · JDK 21
Rendimiento (k6)	`docker compose run --rm k6 run scripts/test.js`	Infraestructura + back arrancados

7.1 Unitarios

```
cd prueba-itx-back
mvn test
```

Clase	Qué cubre
SimilarProductsServiceTest	Orden de similitud, omisión de productos que fallan o no existen, duplicados, errores.
ExistingApisProductCatalogTest	Cliente HTTP contra MockWebServer: 404, 500, timeouts, caché y peticiones concurrentes.
SimilarProductsControllerTest	Contrato JSON, 404, 502 y 401 sin token.
RestExceptionHandlerTest	Traducción de excepciones a respuestas de error.
ProductDetailJsonTest	Formato JSON según el contrato.
ExistingApisPropertiesTest	La configuración se valida al arrancar.

7.2 Integración y cobertura

```
mvn verify
```

SimilarProductsIT levanta la aplicación completa contra mocks; KeycloakSecurityIT arranca un Keycloak real en Docker (Testcontainers) y valida tokens reales. Informes de cobertura (JaCoCo):

```
target/site/jacoco/index.html       (unitarios)
target/site/jacoco-it/index.html    (integración)
```

7.3 End-to-end con Karate

Prueban el sistema desplegado como lo haría un cliente. Antes deben estar arrancados la infraestructura (paso 1) y el back (paso 2).

```
# Con JDK 21 activo (ver apartado 1)
mvn -f e2e/karate/pom.xml test
```

Incluye similar-products.feature (contrato, orden, 404/500, tiempos con productos lentos, caché) y security.feature (sin token, token falso, firma manipulada, token válido, 403 y health público). Informe:

```
e2e/karate/target/karate-reports/karate-summary.html
```

> **Nota.** Si tu Maven usa un repositorio corporativo (Artifactory, Nexus) que no tiene Karate, fuerza Maven Central:
> `mvn -s e2e/karate/central-settings.xml -gs e2e/karate/central-settings.xml -f e2e/karate/pom.xml test`

7.4 Rendimiento con k6

Es el test que proporciona la prueba. Su script obtiene un token de Keycloak antes de empezar y lo envía en todas las peticiones.

```
docker compose run --rm k6 run scripts/test.js
```

Resultados en Grafana: `http://localhost:3000/d/Le2Ku9NMk/k6-performance-test`

8. Tests del front

```
cd prueba-itx-front
npm test           # tests (Vitest + Testing Library)
npm run lint       # análisis estático (ESLint)
npm run build      # compilación de producción en dist/
npm run preview    # sirve la build en http://localhost:4173
```

Fichero	Qué cubre
`utils/cache.test.js`	Caché en cliente: guardado, expiración a la hora y persistencia.
`api/client.test.js`	Caché de peticiones GET, agrupación de peticiones simultáneas, revalidación y POST.
`utils/product.test.js`	Filtrado, formato de precio y normalización de los datos del API.
`auth/AuthProvider.test.jsx`	Login con PKCE, redirección sin sesión, error de Keycloak y logout.
`components/Header/Header.test.jsx`	Logo, contador de la cesta, usuario y cerrar sesión.
`pages/ProductListPage/*.test.jsx`	Listado, búsqueda en tiempo real, enlaces y errores.
`pages/ProductDetailPage/*.test.jsx`	Descripción, breadcrumb, selección por defecto y añadir a la cesta.

> **Nota.** Los tests del front no necesitan Keycloak ni el API: la sesión y las peticiones están simuladas.

9. Resumen de comandos

Qué	Dónde	Comando
Infraestructura	back	`docker compose up -d simulado influxdb grafana keycloak`
Arrancar back	back	`mvn spring-boot:run`
Arrancar front	front	`npm install` y `npm start`
Tests unitarios back	back	`mvn test`
Tests unitarios + integración back	back	`mvn verify`
Tests e2e back (Karate)	back	`mvn -f e2e/karate/pom.xml test`
Test de rendimiento back	back	`docker compose run --rm k6 run scripts/test.js`
Tests front	front	`npm test`
Lint front	front	`npm run lint`
Build front	front	`npm run build`

10. Problemas frecuentes

Síntoma	Causa	Solución
`open //./pipe/dockerDesktopLinuxEngine` al usar docker	Docker Desktop no está arrancado	Abre Docker Desktop y espera a «Engine running»; comprueba con `docker info`.
`NoSuchMethodError: sun.misc.Unsafe.ensureClassInitialized` en Karate	Maven usa un JDK 22 o superior	Activa el JDK 21 (apartado 1) y comprueba con `mvn -v`.
`The JAVA_HOME environment variable is not defined correctly`	JAVA_HOME apunta a una ruta que no existe	Pon la ruta real del JDK, p. ej. `C:\Program Files\Java\jdk-21`.
`Could not find artifact ... karate` en el repositorio corporativo	El Artifactory/Nexus no tiene Karate	Usa los settings de Maven Central (apartado 7.3).
401 al llamar al back	Sin token, token falso o caducado (5 min)	Pide un token nuevo (apartado 5).
El front muestra «No se ha podido iniciar sesión»	Keycloak no está arrancado o aún no está listo	Arráncalo (paso 1), espera unos segundos y recarga.
El front se queda en «Cargando productos...»	El API de Render está arrancando o la red lo bloquea	Espera hasta un minuto; abre `https://itx-frontend-test.onrender.com/api/product` para comprobarlo.
Puerto 8080 ocupado	Keycloak levantado dos veces (back y front) u otra aplicación	Deja un único Keycloak (`docker compose down` en el que sobre).

11. Parar todo

1. Back y front arrancados en terminal: `Ctrl + C` en cada ventana.
2. Contenedores de Docker:

```
cd prueba-itx-back
docker compose down
```